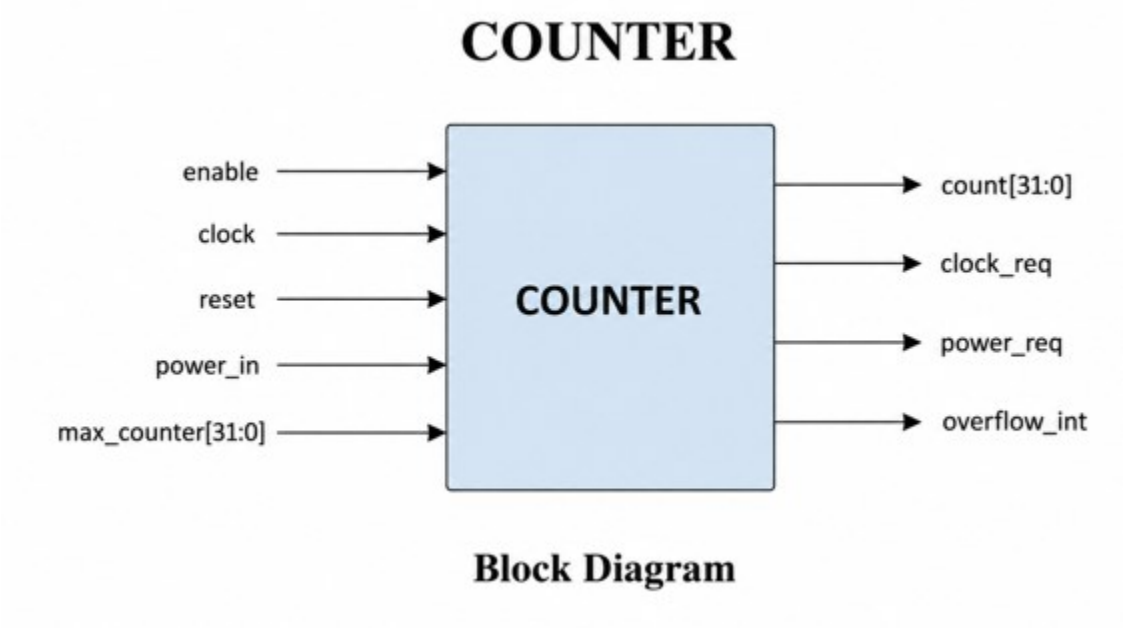
Project Type: SystemVerilog RTL Design and Verification Project

Author: Narasimhamurthy B

1. Project Overview



Block Diagram

The COUNTER module is a configurable 32-bit counter designed using Verilog/SystemVerilog. The design supports enable-controlled operation, power-aware functionality, configurable maximum count control, overflow interrupt generation, and request signaling for clock and power management integration.

A complete self-checking verification environment was developed using SystemVerilog OOP concepts including generator, driver, monitor, scoreboard, functional coverage, interface, and mailbox communication.

2. RTL Features

- 32-bit configurable counter
- Enable-controlled counter operation
- Power-aware counting logic
- Asynchronous reset support
- Programmable maximum count using max_counter input
- Automatic default count value when max_counter = 0

- Overflow interrupt generation
- Clock request signaling
- Power request signaling
- Safe restart behavior after reset/power disable

3. Functional Description

• Counter increments only when enable = 1 and power_in = 1.
• If reset is asserted, count resets to 0 and overflow_int clears.
• If enable or power_in becomes low, counter resets safely.
• When count reaches max_value, overflow_int is asserted and counter resets.
• If max_counter = 0, default value = 50 × 86400 is used.

4. Signal Description

Signal Name	Direction	Width	Description
clock	Input	1	System clock
reset	Input	1	Asynchronous reset
enable	Input	1	Enables counter operation
power_in	Input	1	Indicates power availability
max_counter	Input	32	Programmable maximum count
count	Output	32	Current counter value
clock_req	Output	1	Clock request signal
power_req	Output	1	Power request signal
overflow_int	Output	1	Overflow interrupt flag

5. Verification Environment

The verification environment was developed using SystemVerilog class-based methodology.

Verification Components:
• Generator
• Driver
• Monitor
• Scoreboard
• Functional Coverage
• Interface
• Mailbox Communication

Architecture Flow:
Generator → Driver → DUT → Monitor → Scoreboard/Coverage

6. Verification Methodology

- Generator creates constrained transactions
- Driver drives DUT inputs through virtual interface
- Monitor samples DUT outputs
- Scoreboard compares DUT outputs with reference model
- Coverage component measures functional coverage
- Mailbox communication used between components
- Self-checking verification methodology implemented

7. Functional Coverage

Implemented Covergroups:
• ENABLE_CP
• POWER_CP
• OVERFLOW_CP
• MAX_COUNTER_CP
• ENABLE_POWER_CROSS

Coverage Results:
• Functional coverage achieved: 100%

8. Simulation Results

Observed simulation behavior:
0 → 1 → 2 → 3 → 4 → 5 → overflow → reset

Simulation logs:
PASS: count=0 overflow=0
PASS: count=1 overflow=0
PASS: count=2 overflow=0
PASS: count=3 overflow=0
PASS: count=4 overflow=0
PASS: count=5 overflow=0
PASS: count=0 overflow=1

FINAL COVERAGE = 100%

9. Applications

- Low-power SoC designs
- Embedded controllers
- Watchdog timers
- Event timeout detection
- Power-aware digital systems
- Time-based monitoring systems

10. Conclusion

The COUNTER project successfully demonstrates RTL design and SystemVerilog verification using a self-checking verification environment. The project includes functional coverage, scoreboard validation, waveform analysis, and synthesized hardware schematic verification.

This project demonstrates practical knowledge in:
• RTL Design
• SystemVerilog Verification
• Functional Coverage
• Scoreboard-based checking
• Mailbox communication
• Verification architecture development